

Ejercicios Teóricos y Prácticos con Pilas y Colas

GRADO EN INGENIERÍA DEL SOFTWARE
ESTRUCTURAS DE DATOS

Salvador Sánchez y Sergio Cavero



Universidad
Rey Juan Carlos

Teoría

Ejercicio 1

Supongamos que los elementos A, B, C, D y E se introducen, en ese orden, en una pila inicialmente vacía. A continuación, se extraen cuatro elementos mediante `pop()` los cuales a medida que se extraen se insertan en una cola inicialmente vacía. Si a continuación se retiran dos elementos de la cola, ¿cuál es el siguiente elemento que se retiraría de la cola?

Ejercicio 1 - Solución

PASO 1: Llenar Pila

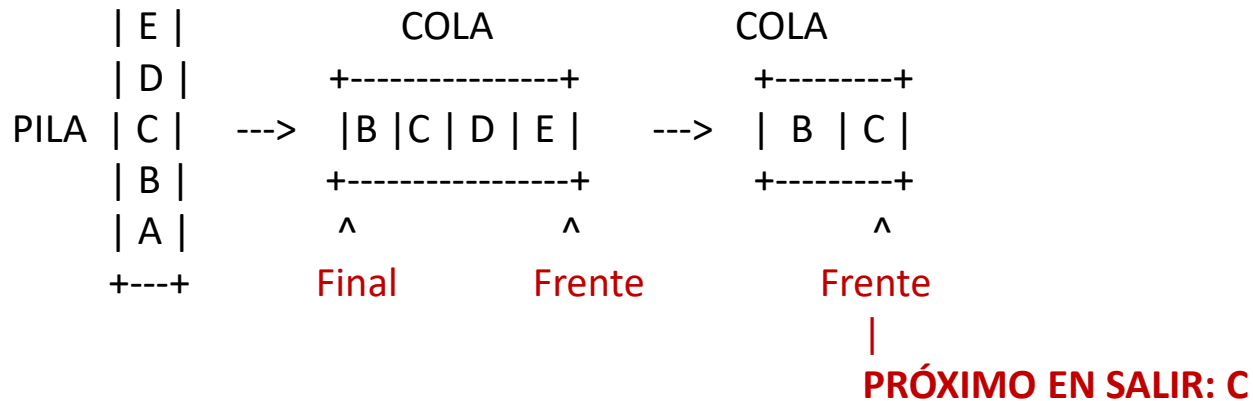
(Push A,B,C,D,E)

PASO 2: Pop Pila -> Push Cola

(E, D, C, B salen)

PASO 3: Dequeue Cola (x2)

(E y D salen)



Ejercicio 2

Describe cómo se debería implementar una Cola utilizando internamente una o varias Pilas, razonando brevemente cómo funcionarían las operaciones básicas en una cola así implementada. A continuación, compara en la tabla proporcionada la complejidad ciclomática de las operaciones básicas (enqueue, dequeue, is_empty) de una Cola convencional frente a tu implementación basada en Pilas.

	Cola convencional	Cola con pilas
enqueue		
dequeue		
is_empty		

Ejercicio 2 - Solución

SOLUCIÓN: Se requieren dos pilas. Una pila principal almacena los elementos en orden de llegada (enqueue). Una pila auxiliar se usa temporalmente para operaciones como dequeue() o first(), que necesitan extraer todos los elementos de la pila para acceder al inicio de la cola y posteriormente, volverlos a añadir a la pila principal.

Alternativamente, la pila auxiliar se puede utilizar al encolar para sacar todos los elementos de la pila, meter el nuevo elemento y devolver los elementos “viejos” de la pila auxiliar a la pila. Para estas operaciones, se transfieren elementos a la pila auxiliar hasta alcanzar el primer elemento de la cola.

	Cola convencional	Cola con pilas (opción 1)	Cola con pilas (opción 2)
enqueue	$O(1)$	$O(1)$	$O(n)$
dequeue	$O(1)$	$O(n)$	$O(1)$
is_empty	$O(1)$	$O(1)$	$O(1)$

Ejercicio 3

Indica si la siguiente afirmación es correcta y justifica tu respuesta:

"La implementación de una función que trabaja con una estructura de datos tendrá siempre la misma complejidad de almacenamiento, se implemente desde la perspectiva del usuario de la estructura de datos o se implemente desde la perspectiva del implementador del TAD de dicha estructura de datos. Podemos por tanto decir que la complejidad de almacenamiento de una operación sobre una estructura de datos es invariante, independientemente de si se implementa desde la interfaz de usuario del TAD (perspectiva de usuario de la TAD) o desde su implementación interna (perspectiva de implementador de la TAD)"

Pon un ejemplo ilustrativo con una pila o una cola.

Ejercicio 3 - Solución

SOLUCIÓN: La afirmación es incorrecta.

La complejidad de almacenamiento Sí varía según la perspectiva de implementación. El usuario del TAD, al no acceder a la estructura interna, carece de optimizaciones disponibles para el implementador.

Por ejemplo, mostrar por pantalla todos los elementos de una pila o una cola como usuario del TAD requiere una estructura de datos auxiliar, lo que duplica el tiempo de procesamiento y aumenta la memoria a $2 \cdot O(n)$. En cambio, el implementador puede optimizar internamente a $O(n)$ usando punteros o referencias, sin estructura auxiliar adicional.

Ejercicio 4

Describe la operación del procedimiento `unknown()`. Demuestra tu comprensión de dicho procedimiento poniendo un par de ejemplos representativos.

```
procedure unknown(var c:tColaEnteros, x:integer);  
begin  
  if (not is_empty(c)) and (first(c) mod x = 0) then begin  
    dequeue(c);  
    unknown(c,x);  
  end;  
end;
```

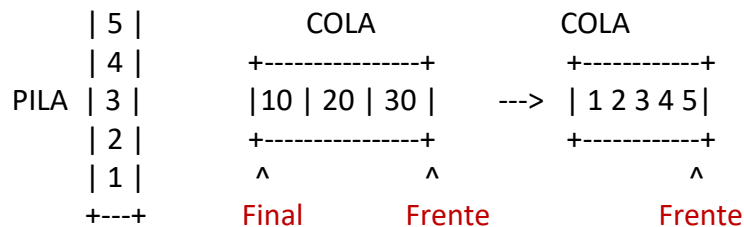
Ejercicio 4 - Solución

```
procedure unknown(var c:tColaEnteros, x:integer);  
begin  
  if (not is_empty(c)) and (first(c) mod x = 0) then begin  
    dequeue(c);  
    unknown(c,x);  
  end;  
end;
```

SOLUCIÓN: El procedimiento elimina de forma recursiva los elementos iniciales de una cola de enteros que sean múltiplos de un valor dado x . Por ejemplo, si inicialmente c es $[6, 12, 5, 24, 8, 3]$, el procedimiento $\text{unknown}(c, 3)$ modificará la cola resultando en $[5, 24, 8, 3]$. El proceso se detiene en cuanto encuentra un número al inicio de la cola que no es múltiplo de x o cuando la cola queda vacía. Si el primer elemento no es múltiplo de x la cola no se verá modificada. Por ejemplo: $[10, 20, 30, 40] \rightarrow \text{unknown}(c, 3) \rightarrow [10, 20, 30, 40]$

Ejercicio 5

Idea un algoritmo que reciba una pila y una cola de enteros e intercambie sus elementos sin alterar el orden, es decir, se deben poner todos los elementos de la pila en la cola y todos los de la cola en la pila sin modificar el orden en que serían extraídos en ambas estructuras.



Complejidad de tiempo esperada: $O(n)$

Espacio auxiliar esperado: $O(1)$

Ejercicio 5 - Solución

SOLUCION:

1. Sacar todos los elementos de la cola y meterlos en la pila, contando en el proceso cuántos elementos se han traspasado.
2. Sacar la pila completa y llevar todos los elementos a la cola.
3. Ahora sacar de la cola tantos elementos como contamos en el paso 1 para sacar solo los que inicialmente estaban en la cola.
4. Tras esto, ya están en el orden original pero los de la pila en la cola y los de la cola en la pila. Y sólo hemos empleado un entero como almacenamiento auxiliar.

Problemas

Problema 1

Escribir un procedimiento `elimina_top_impares(var p:pila)` que, utilizando una unit Pila externa, elimine todos los enteros impares en la parte superior de una pila. Por ejemplo, si el contenido de `p` es `{1,3,5,4,2,3,7,3,5}`, después de `elimina_top_impares(p)` debe quedar `{4,2,3,7,3,5}`.

Restricciones:

- Tiempo de ejecución $O(n)$.
- Optimiza el espacio auxiliar en lo posible.

Problema 1 - Solución

SOLUCIÓN:

```
procedure elimina_top_impares(var p: pila);  
begin  
    while (not is_empty(p)) and (peek(p) mod 2 <> 0) do  
        pop(p);  
    end;
```

Problema 1 - Solución

```
procedure elimina_top_impares_recursoivo(var p: pila);  
{ Una implementación alternativa a la anterior, usando recursividad }  
begin  
  if (not is_empty(p)) and (peek(p) mod 2 <> 0) then  
  begin  
    pop(p);  
    elimina_top_impares_recursoivo(p)  
  end  
end;
```


Problema 2

Implementa un subprograma `eliminar_valor` que elimine todas las ocurrencias de un valor x en una cola de enteros C . Ejemplo: si $C=\{1,3,5,4,2,3,7,3,5\}$, después de ejecutar el subprograma con $x=3$ tendríamos $C=\{1,5,4,2,7,5\}$.

Restricciones:

- El algoritmo debe tener un tiempo de ejecución $O(n)$.
- El subprograma será externo al TAD ColaEnteros por lo que no conocerá su implementación interna. Sí podrás utilizar, sin necesidad de implementarlas, las primitivas del TAD ColaEnteros: `initialize(C)`, `enqueue(C,x)`, `dequeue(C)`, `is_empty(C)` y `first(C)`.

Problema 2 - Solución

```
procedure eliminar_valor(var c:tcola; x:integer);
var
    c_aux: tcola;
    current: integer;
begin
    initialize(c_aux);
    while not is_empty(c) do begin
        { sacamos todos los elementos de la original... }
        current:= first(c);
        dequeue(c);
        if current<>x then enqueue(c_aux,current);
            { guardamos en aux solo los distintos a x }
    end;
    while not is_empty(c_aux) do begin
        enqueue(c,first(c_aux));
        dequeue(c_aux);
    end;
end;
```

Problema 3

Implementa un subprograma rotar que rote a tantos elementos como indique un número n . Consideramos rotar a sacar elementos del inicio de la cola y ponerlos al final-
Ejemplo: si $C=\{10,20,30,40,50\}$, después de ejecutar el subprograma con $n=2$ tendríamos $C=\{30,40,50,10,20\}$.

Restricciones:

- El algoritmo debe tener un tiempo de ejecución máximo $O(n)$.
- Optimiza el espacio auxiliar
- El subprograma será externo al TAD ColaEnteros por lo que no conocerá su implementación interna.

Problema 3 - Solución

```
procedure rotar(var c:cola, n:integer);  
var i, elem: integer;  
begin  
    if not is_empty(c) and n>0 then  
        for i:= 1 to n do begin  
            elem:= first(c);  
            dequeue(c);  
            enqueue(c,elem);  
        end  
    end;  
end;
```

Problema 4

Programa un procedimiento `equivalent_stack(cola,pila)` interno al TAD ColaEnteros que reciba una cola y retorne una pila con los mismos elementos de la cola pero apilados, teniendo en cuenta que el orden de entrada de los mismos en la cola debe ser el mismo que en la pila resultante.

Por ejemplo, si la cola de entrada contiene los elementos: `inicio -> 1, 2, 3, 4, 5 <- final`, después de ejecutar el procedimiento `equivalent_stack(c,p)`, la pila `p` deberá contener los elementos apilados de la siguiente manera: `fondo -> 1, 2, 3, 4, 5 <- cima`.

Actualmente el TAD ColaEnteros está internamente implementado mediante una estructura con nodos y dos punteros: `first` y `last`, e incluye las 5 operaciones básicas. También tienes disponible el TAD PilaEnteros (no tienes que implementarlo) con sus 4 operaciones básicas.

Importante: El estado final de la cola recibida como parámetro debe ser idéntico al estado inicial al finalizar el procedimiento.

Problema 4 - Solución

```
procedure equivalent_stack(var c: tCola; var p: tPilaEnteros);  
var  
    node: ^nodo;  
begin  
    initialize(p);  
    if not empty_queue(c) then begin  
        node := c.first;  
        while node <> nil do begin  
            push(p, node^.info);  
            node := node^.sig;  
        end;  
    end;  
end;
```